

AirPulse 2.0 Platform

Comprehensive Technical Documentation & System Reference

Executive System Overview

AirPulse 2.0 is a production-quality, premium Environmental Intelligence Platform. It acts as a hybrid educational tool and a real-time tracking interface, highlighting the synergistic power of relational database systems (SQLite) combined with NoSQL cloud document stores (MongoDB Atlas). The platform is engineered to display modern full-stack developer patterns while preserving and demonstrating core academic DBMS principles (normalization, relational mapping, foreign keys, active triggers, indexed searches, and database views) required for college assessments.

Core Technical Pillars

- **Primary Relational Layer (ACID):** Hosted entirely on SQLite (airpulse_relational.db), enforcing strict constraints, foreign key referential integrity (CASCADE deletions), and indexing.
- **Secondary Mirroring Layer (NoSQL):** Replicated in real-time to MongoDB Atlas cloud database cluster, utilizing dynamic mirroring functions during mutations to sync stations, logs, and alert feeds.
- **Real-Time Eventing Service:** Socket.IO bridges frontend UI updates instantly when database writes occur—enabling immediate UI updates and audit logs without web refreshes.
- **User Session Persistence:** Local caching using localStorage ensures query history, custom Mongo templates, and SQL playground execution state survive page updates.
- **Robust Fallback Capabilities:** Multi-tier offline safety parameters handle geocoding failures via SQLite cache logs and database failures via JSON NoSQL mock emulators.

1. Technology Stack & Tool Integrations

AirPulse 2.0 leverages a highly-scalable stack. The system dependencies and tools have been selected to ensure offline presentation stability while preserving capability to connect to cloud services.

Component	Technology / Tool Used	Functional Purpose
Client Interface	React (JS), Vite, Tailwind CSS	Builds dynamic interface with CSS animations
Backend Server	Python, Flask, Flask-SocketIO	Exposes REST API gateways and event channels
Relational DBMS	SQLite 3 (Primary source of truth)	Manages normalization, triggers, constraints, views
NoSQL DBMS	MongoDB Atlas (Cloud Cluster DB)	Synchronized secondary mirror layer for document queries
3D Globe Visualization	react-globe.gl & Three.js	Renders interactive gl canvas for exploration
AI Advisory	Google Gemini 1.5 Flash API	Constructs sensitive group health advisories
Uptime Weather	OpenWeatherMap API	Displays local temperature, humidity, and atmospheric state
Analytics Feed	AQICN WAQI API Feed	Syncs live air quality metrics from global monitors
Report Exporter	ReportLab, Pandas, OpenPyXL	Compiles customized PDF audits, CSV sheets, and Excel logs

System Architecture Flow

A typical database mutation flows through the architecture in the following order:

1. A client initiates a mutation (e.g., Inserting a record or adding a station node).
2. The Flask controller targets the SQLite primary database via Python's sqlite3 driver.
3. SQLite applies integrity checks (foreign keys, CHECK bounds) and executes associated **SQL Triggers**.
4. The SQLite database inserts the data, auto-logging the action into the AuditLog table and compiling alerts into the SystemAlerts table.
5. The backend mirroring functions retrieve these tables and update the MongoDB Atlas collections.
6. An event is broadcasted via **Socket.IO**, triggering a toast message on client interfaces and causing the dashboard to refresh metrics seamlessly.

2. Core Features & Functional Description

The application divides operations into distinct tabs within the user interface, serving specific functional domains:

Real-time Dashboard Console

Displays current atmospheric metrics dynamically. Includes an animated circular AQI Gauge, historical 7-day trend charts, active system safety alert feeds, weather parameters, and dynamic health recommendation advisories.

Immersive spatial views

Contains a 3D Earth Globe (React-Globe.gl) supporting rotation, zoom, country boundaries hover, and geocoded monitor node placements, alongside an SVG Vector Map (IndiaMap) with zoom behaviors.

SQL Playground Console

Allows executing custom queries directly on SQLite. Exposes a split text editor and pagination grid with vertical/horizontal scrollbars, client filter filters, Excel/CSV exporters, and localStorage persistence.

NoSQL Mongo Console

Supports executing MongoDB statements using BSON. Outputs formatted documents in collapsible tree-views, history logs list dropdown, custom-saved templates sidebar, document copy controls, and pagination.

Academic DBMS Center

Academic guidelines featuring ER schemas, live table schemas, 1NF/2NF/3NF/BCNF normalization tables, SQL triggers documentation, database views definitions, and an interactive Viva prep question board.

User Journey Persistence Analytics

Integrated dashboard widgets: 'Favorite Places' (adds/removes favorited locations with selection drill-down), 'Recently Explored' (lists last 10 searches), and 'Trending Insights' (most searched cities/countries, severe alerts, highest AQI).

Dynamic Environmental Gamification

Checklist of green actions (mobility, energy saving, greenery). Completing actions awards points (XP) which dynamically unlocks visual achievements and badges (Zero Carbon, Clean Air, Seed Sower).

3. Database Schema, Constraints, & Triggers

The project implements a normalized relational database design satisfying 3NF. Foreign key constraints enforce referential boundaries, and active triggers automate audits.

Relational Schema Definitions

- **Stations:** Stores monitor locations.

Columns: StationID (PK), Name (Unique), City, Latitude, Longitude, Status (CHECK: Active, Inactive, Maintenance), EstablishedDate.

- **HealthAdvisories:** Reference lookup for health precautions.

Columns: CategoryName (PK), AQILower, AQIUpper, GeneralAdvisory, ChildElderlyAdvisory, ActionTip, ColorCode.

- **PollutionRecords:** Daily pollutant logs.

Columns: RecordID (PK), StationID (FK referencing Stations ON DELETE CASCADE), Timestamp, PM25, PM10, CO, NO2, SO2, O3, AQI, AQI_Category (FK referencing HealthAdvisories).

- **SystemAlerts:** Exceeded pollutant indicators.

Columns: AlertID (PK), StationID (FK), RecordID (FK ON DELETE CASCADE), Pollutant, ObservedValue, ThresholdValue, AlertTimestamp, Status (CHECK: Active, Resolved).

- **AuditLog:** Auto-populated historical audit journal.

Columns: LogID (PK), ActionType (CHECK: INSERT, UPDATE, DELETE), TableName, RecordID, OldValues (JSON), NewValues (JSON), ActionTimestamp, ExecutedBy.

Relational Triggers

The database handles business logic on the database side using four active triggers:

1. **trg_pollution_after_insert:** Fired on new records. Automatically writes a log entry into the AuditLog table. It also scans concentrations: if PM2.5 exceeds 150.0, SO2 exceeds 75.0, or AQI exceeds 200.0, it generates active warning rows in SystemAlerts.
2. **trg_pollution_after_update:** Triggered when record concentrations are modified. Logs the changes (storing old and new states in JSON format) and resolves existing alerts if the updated AQI drops below safe thresholds.
3. **trg_pollution_after_delete:** Triggered on row removal, logging old metrics to the audit journal.
4. **trg_stations_after_insert:** Logs the registration of new station nodes into the audit log automatically.

SQL Trigger Demonstration (Insert Auditing)

```
CREATE TRIGGER trg_pollution_after_insert AFTER INSERT ON PollutionRecords BEGIN
    INSERT INTO AuditLog (ActionType, TableName, RecordID, NewValues)
    VALUES ('INSERT', 'PollutionRecords', NEW.RecordID,
    '{"StationID":'||NEW.StationID||',"AQI":'||NEW.AQI||'}');
END;
```

4. Dual-Database Mirroring & Real-Time Sync

A critical feature of AirPulse 2.0 is its dual-database design. It utilizes SQLite for ACID compliance and academic evaluation, and MongoDB Atlas for real-time unstructured cloud storage.

MongoDB Replicated Collections

Every write operation on SQLite calls a synchronizer. The system flushes and replicates records into these primary MongoDB Atlas collections:

- **stations**: Replicates coordinate documents, names, statuses, and latest readings.
- **pollution_records**: Maps relational records with key pollutant components.
- **system_alerts**: Syncs active warning records.
- **audit_logs**: Mirrors SQLite mutation audits.
- **search_history**: Persists user-searched city geocodes, timestamps, resolved AQIs, and click sources.
- **favorite_locations**: Stores custom locations favorited by users in the side drawer.
- **location_interactions**: Logs click and search telemetry logs for analytics.

WebSocket Broadcast Event Propagation

Socket.IO handles event updates instantly across active clients. When database writes or sync events compile, the server fires a `db_mutation` event containing details. Clients intercept this websocket event, trigger a synchronization toast notification, and dynamically pull updated stats without reloading the application.

Robust Network Fallbacks

*To guarantee deployment reliability during internet outages or server issues, the connection engine implements self-healing fallbacks. If MongoDB Atlas is unavailable or credentials fail, the server dynamically initializes an offline **Mock NoSQL Emulator (JSON storage)**. The dashboard remains online and responsive, saving state locally in `mock_mongodb.json`.*

5. SQL/NoSQL Playgrounds & Offline Caching

To maximize user experience and system resilience, AirPulse 2.0 incorporates advanced console upgrades, complete session persistence, and self-healing fallback mechanisms.

SQL Playground Usability Upgrades

The SQL console has been upgraded to a production-grade query grid featuring:

- **Scrollable Container & Sticky Headers:** Decoupled results tables support vertical and horizontal scrolls with sticky headers.
- **Interactive Pagination:** Supports page size configurations of 10, 25, 50, or 100 rows with navigation buttons.
- **Query Filter Input:** On-the-fly search input filters matching cells instantly.
- **Client-Side Blob Exports:** Allows fast downloads to CSV and XML-formatted Excel spreadsheet files.
- **localStorage Persistence:** Caches editor query, result sets, current page index, size, and filters across session reloads.

MongoDB Tree-View Playgrounds

Query output is formatted as an interactive tree-view structure, enabling users to expand and collapse document fields. The NoSQL panel features command history (last 10 queries), custom-saved template shortcuts, individual document copy-to-clipboard utilities, and paginated BSON results.

Offline API Fallback Cache Lookup

If external geocoding (WAQI/AQICN) or weather APIs encounter network failures or rate limits, the explore engine fallbacks to the local SQLite database. It performs queries matching the searched city/coordinates. It retrieves the matching record, maps it with the `offline: true` parameter, and displays the fallback cached dataset inside the client drawer accompanied by a warning banner outlining the `lastUpdated` timestamp.

User Session Intelligence Widgets

Three widgets display exploration analytics live on the dashboard:

1. **Favorite Places:** Displays saved favorite cities, with drill-down exploration queries and inline deletion triggers.
2. **Recently Explored:** Lists the last 10 geocoded queries and their resolved AQIs.
3. **Trending Insights:** Auto-computes analytics (most searched cities/countries, peak AQI value, and severe warning logs).